Computación gráfica y ambientes virtuales (0494)

Práctico 08

Objetivo

El objetivo principal es mejorar la apariencia de la escena: que el terreno pueda tener aspecto de suelo y darle mayor profundidad a la escena con la incorporación de un fondo que tenga la apariencia del cielo. Las dos cosas se pueden resolver con texturas. Hasta acá el color de cada objeto era un único color plano o un valor por vértice que, al momento de digitalizar el polígono y rellenarlo, se interpolaba; con una textura, el color se determina punto por punto sobre la superficie del polígono leyéndolo desde una imagen almacenada en la memoria de la GPU.

Como objetivo secundario hay que hacer una verificación que viene desde el Práctico 03: la tupla de vértices tiene coordenadas de textura que nunca se validaron. En este práctico se necesitan las mismas para poder usar una textura para cambiar la apariencia de un objeto por lo que hay que verificar que el cálculo de las mismas sea correcto y logre la apariencia deseada.

Punto de partida

La escena del Práctico 07: el terreno como malla indexada, con el recorte de cara trasera activado y pintado de un color plano. OpenGL no tiene soporte para la manipulación de imágenes por lo que hace falta incorporar una librería externa; se propone usar `stb_image.h` (https://github.com/nothings/stb); se trata de una biblioteca de un único archivo de cabecera: se copia al proyecto y se incluye igual que `FastNoiseLite.h` del práctico anterior. En el aula virtual quedan dos texturas de suelo y un cubemap con imágenes de un cielo para poder hacer las pruebas; también puede usarse cualquier otra imagen.

Parte 1 - El módulo `Texture`

Las texturas en OpenGL se trabajan igual que los buffers utilizados para pasar la información de los vértices al GPU: hay que reservar espacio en memoria y al terminar de usarlo hay que liberarlo. Requieren, por lo tanto, el mismo tratamiento en cuanto a quién es el administrador de esta memoria y cómo se encarga de liberarla (copia y destrucción). Es por esto que conviene encapsular la administración de una textura en un objeto que sea dueño del identificador que devuelve OpenGL (lo mismo que se hizo para la clase Mesh). No es recomendable tener código suelto relacionado con administración de texturas en todas partes del proyecto.

Responsabilidades

- Ser el único dueño del identificador: lo libera al destruirse, no se puede copiar y sí se puede mover. Son las mismas tres reglas utilizadas para construir `Mesh`.
- Crear la textura mediante las cuatro etapas: crear, reservar, llenar y parametrizar usando el estilo DSA (no usar `glGenTextures` ni `glBindTexture`).
- **No leer archivos.** Leer y decodificar la imagen es C++ puro y la responsabilidad ya la tiene el módulo `ResourceManager`. Hay que agregar ahí un `struct` por tipo de recurso junto con dos métodos `load_*` y `get_*` para el tipo de recurso.

Por dónde empezar

Si la imagen se encuentra almacenada en el disco, la decodificación es una tarea del lado del CPU y OpenGL no interviene para nada en esto (puede estar también la posibilidad de generar la imagen de

manera procedural tanto en el CPU como en el GPU). Por otra parte la *textura* sí es un recurso de OpenGL que se administra del lado de la GPU (es la misma separación que existe entre `MeshData` y `Mesh`). Por lo tanto lo primero que hay que hacer es leer y decodificar la imagen para poder pasarla a la memoria de la GPU. En el siguiente fragmento de código se muestra una propuesta de interfaz para esto; la lectura y decodificación de la imagen se hace usando `stb_image` en la implementación de la interfaz.

```
// En ResourceManager.h - C++ puro, no hace falta OpenGL
struct ImageData
{
    int width    = 0;
    int height   = 0;
    int channels = 0;                    // se obtiene del ARCHIVO, informativo
    std::vector<unsigned char> pixels;   // siempre RGBA: 4 bytes por texel
};

// Dentro de la clase ResourceManager:
...
const ImageData& load_image(const std::string& key, const std::string& file, bool flip_vertically);
const ImageData& get_image(const std::string& key) const;
...
```

Tener en cuenta

- `pixels` se guarda siempre como `RGBA` aunque el archivo tenga tres canales. Hay que fijar el último argumento de `stbi_load(...)` en 4 para que los datos decodificados de la imagen estén en este formato. Esto permite que el código que sube la imagen al GPU no dependa de la cantidad de canales del archivo (decisión de diseño que no es la única posible).
- Hay que declarar un `#define STB_IMAGE_IMPLEMENTATION` en **un solo** `.cpp` de todo el proyecto (en `ResourceManager` es el lugar natural). Si se declara en más de un lugar, el linkeador dará error de símbolos duplicados. Ver ayuda de `stb_image`.

Una vez que las funcionalidades para decodificar una imagen están listas, se puede pasar al desarrollo de la clase que administra una textura. El fragmento de código a continuación es una propuesta para la interfaz de dicha clase.

```
// Texture.h - punto de partida, no molde
// Dueño unico de una textura de OpenGL. Mismas reglas que Mesh: el destructor libera, la copia esta
// prohibida, mover traspasa la propiedad.
class Texture
{
public:
    Texture() = default;
    ~Texture();

    Texture(const Texture&) = delete;
    Texture& operator=(const Texture&) = delete;
    Texture(Texture&& other) noexcept;
    Texture& operator=(Texture&& other) noexcept;

    // Los cuatro pasos (crear, reservar, llenar, parametrizar) para una
    // textura 2D, a partir de una imagen ya decodificada.
    void load_2d(const ImageData& image);

    // Lo mismo para un cubemap: seis caras en el orden +X -X +Y -Y +Z -Z.
    void load_cubemap(const std::array<const ImageData*, 6>& faces);

    // Conecta la textura a una unidad de textura para el proximo dibujo.
    void bind(unsigned int unit) const;

    void clear(void);
    unsigned int id(void) const { return id_; }

private:
    unsigned int id_ {0U};
};
```

Implementación de la carga de una textura 2D

- Escribir `Texture::load_2d` con las cuatro etapas de la filmina *Las cuatro etapas usando DSA*. La cantidad de niveles de mipmap se calcula como $\lfloor \log_2(\text{máx}(w, h)) \rfloor + 1$, es decir, 9 para una imagen de 256 pixeles y 11 para una de 1024 pixeles.
- Los parámetros de muestreo dependen del uso de la textura. Para el terreno de la Parte 2 conviene `GL_LINEAR_MIPMAP_LINEAR` para la reducción, `GL_LINEAR` para la ampliación y `GL_REPEAT` en las direcciones `S` y `T`, pero otra textura puede necesitar otros; por eso conviene que la clase permita configurarlos (ver la última de las decisiones de diseño a continuación).

Decisiones de diseño a tomar

- ¿`ResourceManager` libera la imagen decodificada una vez que se creó la textura? Al guardarla en su caché (lo que evita volver a leer el disco si se pide dos veces) la imagen queda en la memoria del CPU durante todo el programa, aunque la GPU ya tenga su copia: una imagen de 1024×1024 en `RGBA` ocupa 4 MiB. Se puede ofrecer un método para liberarla (por ejemplo, `unload_image`), con el costo de tener que leer el disco otra vez si se la vuelve a pedir.
- ¿Cuándo se decide si la imagen se invierte verticalmente: al momento de cargarla o al momento de usarla? Una textura 2D necesita invertirse mientras que caras de un cubemap no (Parte 4). La firma propuesta lo hace durante la carga; la alternativa es invertir la componente v en el *shader*.
- ¿Una sola clase para los dos tipos de textura o dos clases?
- ¿Los parámetros de muestreo (filtros, repetición, etc.) quedan fijos al crear la textura o se pueden cambiar después?

Parte 2 - El terreno con textura

Coordenadas de textura

Se pueden obtener al construir la grilla sin calcular nada nuevo: los índices (i, j) del nodo determinan su posición en la malla por lo que al dividir por N se obtienen las coordenadas (u, v) para la textura en el intervalo $[0, 1]$. En el mismo bucle donde se construyen los vértices de la malla (Parte 1 del Práctico 07) se puede hacer lo siguiente:

```
Vertex v;
v.position   = { j * paso, altura(i, j), i * paso };
v.tex_coords = { float(j) / N, float(i) / N };    // [0,1] sobre toda la malla
vertices.push_back(v);
```

Sin embargo, con eso la textura se aplica sobre todo el terreno y queda "estirada": con una ventana de terreno de 2 km de lado y una imagen de 256 texels, cada texel mide casi 8 metros. Para obtener un efecto más suave en la apariencia del terreno usando una sola textura se puede hacer que la textura se *repita* sobre la malla. Para ello hay que multiplicar las coordenadas (u, v) en el *shader* por un factor de escala y usar el modo `GL_REPEAT` al crear la textura. Con esto OpenGL maneja de forma automática las coordenadas que se salen del rango $[0, 1]$ y se logra aplicar la textura a superficies más pequeñas de la malla del terreno.

En la propuesta a continuación para el código del *vertex* y *fragment* shader para la malla del terreno, se propone un `uniform` que controle la cantidad de veces que se repite la textura sobre la malla del terreno.

```
// terreno.vs
#version 460 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;      // todavia no la lee nadie
layout (location = 2) in vec2 aTexCoord;

uniform mat4 uModel;
uniform mat4 uView;
uniform mat4 uProjection;
uniform float uTeselas;      // cuantas veces se repite la textura en la malla

out vec2 vTexCoord;

void main()
{
    vTexCoord   = uTeselas * aTexCoord;     // [0,1] -> [0,uTeselas]
    gl_Position = uProjection * uView * uModel * vec4(aPos, 1.0);
}

// terreno.fs
#version 460 core
in vec2 vTexCoord;

layout (binding = 0) uniform sampler2D uTextura;   // unidad 0

out vec4 FragColor;

void main()
{
    FragColor = texture(uTextura, vTexCoord);      // REEMPLAZA el color
}
```

Con `layout (binding = 0)` el *sampler* queda conectado a la unidad 0 sin ningún `uniform` del lado de C++: alcanza con `bind(0)` antes de dibujar. Si se prefiere fijarlo desde C++, el valor que lleva el `uniform` es el **número de unidad**, no el identificador de la textura.

Ajustar la apariencia del terreno con la textura

- Configurar el muestreo de la textura del terreno con los parámetros indicados en la Parte 1.
- Elegir el número de teselas y dejar escrito cuántos metros cubre cada una. Como referencia, si se toman 40 teselas sobre 2 km se obtienen 50 m por tesela.

La trampa de los mipmaps

Si se reservan varios niveles para los mipmaps usando `glTextureStorage2D` y, luego, no se generan con `glGenerateTextureMipmap`, OpenGL no genera ningún error: los niveles existen pero su contenido es indefinido. El síntoma es que de cerca la textura se ve bien, o con ruido de colores, y hacia el horizonte aparece un color que no está en la imagen. Conviene probarlo una vez a propósito para poder identificar el problema en el futuro.

Parte 3 - Las coordenadas de las primitivas

Las primitivas del Práctico 03 (cubo, cilindro y cono) calculan coordenadas para textura pero hasta acá nunca se usaron. Si alguna está mal, no hubo forma de notarlo: no hay error de compilación, no hay mensaje de OpenGL y los píxeles se pintan con colores sólidos.

Cómo hacer la verificación

Dibujar cada primitiva aplicando una textura y hacer una inspección visual en la pantalla. La imagen a usar para la textura tiene que ser *asimétrica en los dos ejes*, por ejemplo: una letra, un número, un texto; si se utiliza un damero (tablero de ajedrez) no es posible distinguir una textura invertida o espejada de una que es correcta.

Dos cosas que van a aparecer y **no son errores**:

- **El cono** muestra la textura estirada en forma radial, desde el vértice hacia el borde de la base. Esto se debe a que sus coordenadas de textura se calculan proyectando cada vértice sobre el plano

de la base, mientras que las del cilindro se obtienen "desenrollando" su superficie lateral sobre un rectángulo. Son dos formas distintas de asignar coordenadas que no están mal (dependen de cómo se quiera aplicar la textura).

- **La costura del cilindro**: al desenrollar la superficie lateral, la última columna de vértices ocupa la misma posición que la primera pero con $u = 1$ en lugar de $u = 0$, por lo que esos vértices están repetidos en la malla. Si se usara un único vértice con $u = 0$ para cerrar el cilindro, el último gajo interpolaría u desde casi 1 hasta 0 y mostraría la textura entera comprimida y al revés.

Decisión de diseño a tomar

Hasta acá cada pieza de la aeronave se dibuja con un color. En la organización propuesta en el Práctico 04, el `RenderItem` que entrega `Aircraft::collect()` lleva la malla, la matriz de modelo y el color; otros diseños tendrán una organización de código equivalente. Al agregar texturas, el código de dibujo necesita saber además qué textura usar para cada pieza (o si la pieza sigue usando un color plano). Hay varias formas de resolverlo, por ejemplo: agregar a esa información un puntero a una `Texture` que no es dueño de ella (con `nullptr` para indicar color plano), un identificador de la textura, o un shader distinto para cada tipo de pieza.

Parte 4 - Fondo de la escena: el cielo

Una forma de generar el fondo de la escena es usar un cubo que rodea al observador (posición de la cámara) y que se dibuja en la profundidad máxima de la escena (detrás de todo). Sobre las caras de este cubo se aplican seis imágenes (una por cara) que muestran un paisaje de lejanía, por ejemplo: horizonte lejano, montañas lejanas, cielo con nubes. Con esta técnica se logra una ilusión óptica de profundidad o lejanía porque al no haber paralaje el ojo no tiene forma de medir la distancia del cubo y el cerebro lo interpreta como "en el fondo" de la escena (la ilusión no está asociada al tamaño del cubo).

La textura que se utiliza para esto se denomina *cubemap*. Posee una representación especial en memoria porque ya está preparada para almacenar las seis imágenes de las caras del cubo y el muestreo de las mismas se realiza por *dirección* respecto al observador y no por coordenadas (u, v) como en las texturas clásicas.

Crear una textura tipo *cubemap*

El almacenamiento se reserva como si fuese una textura **2D** pero se deben cargar seis imágenes (una por cara) como si fuese una capa de una textura **3D**. El índice de la capa indica a qué cara del cubo corresponde cada imagen, con el siguiente orden: $0 = +X$ (derecha), $1 = -X$ (izquierda), $2 = +Y$ (arriba), $3 = -Y$ (abajo), $4 = +Z$ (frente) y $5 = -Z$ (atrás). Es el mismo orden que el de las constantes `GL_TEXTURE_CUBE_MAP_POSITIVE_X` a `GL_TEXTURE_CUBE_MAP_NEGATIVE_Z`; en los archivos de un cubemap las caras suelen llamarse `right`, `left`, `top`, `bottom`, `front` y `back`, en ese orden.

Las caras **no** se invierten al leerlas, al revés que la textura del terreno. La razón es que OpenGL ubica el origen de una textura 2D en la esquina inferior izquierda de la imagen, mientras que las caras de un cubemap usan una convención (heredada de RenderMan) con el origen en la esquina superior izquierda, que coincide con el orden en que `stb_image` entrega las filas de la imagen.

```
// Texture::load_cubemap. Las caras llegan SIN invertir (al reves que en 2D).
void Texture::load_cubemap(const std::array<const ImageData*, 6>& faces)
{
    clear();                          // el almacenamiento no se reserva dos veces
    const int w = faces[0]->width, h = faces[0]->height;
    glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &id_);
    glTextureStorage2D(id_, 1, GL_RGB8, w, h);               // se reserva en 2D...
    for (int cara = 0; cara < 6; ++cara)
        glTextureSubImage3D(id_, 0, 0, 0, cara, w, h, 1,     // ...y se llena en 3D:
                            GL_RGBA, GL_UNSIGNED_BYTE,       // la cara es la capa de la textura
                            faces[cara]->pixels.data());
    glTextureParameteri(id_, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTextureParameteri(id_, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glTextureParameteri(id_, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);   // en las tres direcciones
    glTextureParameteri(id_, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
    glTextureParameteri(id_, GL_TEXTURE_WRAP_R, GL_CLAMP_TO_EDGE);
}
```

De acuerdo con la convención a usar para decodificar los datos de imagen propuesta en el Punto 1, los datos llegan como `RGBA`, sin embargo, en el código de muestra de arriba las texturas del cubemap se almacenan en la memoria de la GPU como `GL_RGB8`. Esto no es un problema porque el formato de los datos y el del almacenamiento no tienen por qué coincidir, OpenGL se encarga de convertir al formato de destino.

En las opciones de tratamiento de borde de la textura se utiliza la opción `GL_CLAMP_TO_EDGE` en lugar de `GL_REPEAT` para evitar que el algoritmo de filtrado tome texels del borde opuesto de cada cara que haría que se distingan líneas de cambio de imagen en las aristas del cubo.

Muestreo de un *cubemap*

Como se mencionó antes, en una textura tipo *cubemap*, el muestreo se realiza por dirección (vector de 3 componentes) y no por par (u, v). Esta dirección se obtiene mediante la posición del vértice medida desde el centro del cubo, por lo que, si el sistema de referencia del cubo está en el centro del mismo, la posición de cada vértice es la dirección que se tiene que usar para muestrear. El cubo tiene que estar centrado en el origen de su propio sistema de coordenadas (la primitiva del cubo del Práctico 03 ya lo está). En cambio, la dirección no necesita ser unitaria: para muestrear, OpenGL elige la cara según la componente de mayor valor absoluto y divide las otras dos por ella, por lo que el tamaño del cubo no influye.

Observador que se desplaza por la escena

Lo planteado hasta acá para el cubo como fondo de la escena asume que el observador está fijo en la misma: puede rotar pero no puede desplazarse. Si el observador debe desplazarse, entonces, hay que mover el cubo con el observador porque sino, dependiendo del tamaño que se haya elegido para el cubo, puede suceder que el observador alcance (y sobrepase) las caras del mismo y "salga" de la escena.

Desde el punto de vista matemático lo que hay que hacer es agregar la misma transformación de traslación del observador al cubo que representa el fondo. Esto es equivalente a *eliminar la traslación* de la matriz de vista que se utilice para dibujar el cubo. Recordar que la matriz de vista contiene la posición de la cámara en el mundo y que se usa para transformar el mundo a la posición canónica; si una vez que el mundo está en la posición canónica, agregamos el cubo centrado en el origen, el efecto será el mismo que si el cubo se hubiese desplazado con el observador.

Computacionalmente esto es muy barato porque no hay que calcular una transformación nueva para el cubo en cada cuadro, solamente hay que copiar la matriz de vista y eliminar la componente de traslación de la misma. Esta matriz de vista "modificada" es la que se pasa al shader que se usa para procesar el cubo del fondo de la escena.

Shaders para el fondo

A continuación se muestra una propuesta de los programas *vertex* y *fragment* shader para dibujar el fondo de una escena usando un cubo con una textura tipo cubemap.

```
// background.vs
#version 460 core
layout (location = 0) in vec3 aPos;

uniform mat4 uProjection;
uniform mat4 uView;          // la vista sin traslacion

out vec3 vDireccion;

void main()
{
    vDireccion  = aPos;      // desde el centro, cada vertice es una direccion
    gl_Position = uProjection * uView * vec4(aPos, 1.0);
}

// background.fs
#version 460 core
in vec3 vDireccion;

layout (binding = 0) uniform samplerCube uCielo;

out vec4 FragColor;

void main()
{
    FragColor = texture(uCielo, vDireccion);
}
```

Organización del dibujo del lado del CPU

Finalmente, para enviar el cubo al pipeline del GPU y que se dibuje en el "fondo" de la escena hay dos métodos diferentes que no deben mezclarse:

(a) Dibujar el fondo *primero*, sin escribir en el buffer de profundidad (z-buffer).
(b) Dibujar el fondo al *último*, ajustando su profundidad en el shader.

A continuación se muestra un fragmento de código para dibujar el fondo usando el método (a). En este caso se logra que el cubo quede en el fondo porque se dibuja primero y con la escritura en el buffer de profundidad desactivada (`glDepthMask(GL_FALSE)`). El test de profundidad sigue activo, pero como el buffer acaba de inicializarse con el valor máximo, todos los fragmentos del cubo lo pasan y el cubo llena el framebuffer como si fuese el color con el que se inicializa. El buffer de profundidad, en cambio, queda con el valor máximo porque el dibujo del cubo no escribe en él. Luego, antes de dibujar la escena, se vuelve a activar la escritura para que el resto de la escena se dibuje en el orden correcto sobre el fondo que ya se pintó.

```
// Metodo (a): el fondo PRIMERO, sin escribir profundidad.
const glm::mat4 vista_sin_traslacion = glm::mat4(glm::mat3(vista));   // eliminar traslacion de la vista
glDepthMask(GL_FALSE);                                                // no escribir en el z-buffer
glDisable(GL_CULL_FACE);                                              // el cubo se mira desde adentro
glUseProgram(prog_fondo);
glProgramUniformMatrix4fv(prog_fondo, loc_projection, 1, GL_FALSE,    // uProjection
                          glm::value_ptr(proyeccion));
glProgramUniformMatrix4fv(prog_fondo, loc_view, 1, GL_FALSE,          // uView
                          glm::value_ptr(vista_sin_traslacion));
textura_fondo.bind(0);                                                // el cubemap, unidad 0 (uCielo)
glBindVertexArray(cubo_fondo.vao());
glDrawElements(GL_TRIANGLES, cubo_fondo.count(), GL_UNSIGNED_INT, nullptr);
glDepthMask(GL_TRUE);                                                 // volver a escribir en el z-buffer
glEnable(GL_CULL_FACE);                                               // recorte de caras traseras
// ... y recien ahora, la escena
```

En el método (b) se dibuja primero toda la escena y al final el fondo. Para que el fondo quede detrás de

todo se fuerza su profundidad al valor máximo desde el *vertex* shader: se calcula la posición proyectada del vértice, `vec4 pos = uProjection * uView * vec4(aPos, 1.0)`, y se asigna `gl_Position = pos.xyww`. Como la coordenada z se iguala a la coordenada homogénea w, después de la división por w la profundidad de todos los fragmentos del cubo vale 1, que es la del plano lejano. Ese es el mismo valor con el que se inicializa el buffer de profundidad, por lo que hay que cambiar la comparación del test con `glDepthFunc(GL_LEQUAL)` (menor o igual) para que el fondo pase el test justamente en los píxeles donde la escena no dibujó nada. Con esto, además, no se calcula el color del fondo en los píxeles que ya están tapados por la escena. Al terminar hay que volver a la comparación por omisión, `glDepthFunc(GL_LESS)`.

La malla del cubo (`cubo_fondo`) se puede obtener simplemente de las primitivas del Práctico 3 y luego se le aplica una escala apropiada para la escena que se esté modelando. Para ello hay que tener en cuenta que, si bien el tamaño no influye al momento de proyectar el fondo y el cubo se mueve con el observador, cuando la geometría del mismo se procese en el pipeline de la GPU se verá afectado por las operaciones de recorte y por lo tanto tiene que quedar entre los planos del volumen de visualización *cercano* (near) y *lejano* (far). Por ejemplo, un cubo de lado 2 en una escena con el plano cercano en 1 queda cortado y el cielo aparece como una franja. Con el método (b) este problema no aparece, porque la profundidad del cubo se fuerza al plano lejano.

Recorte de caras traseras y geometría del fondo
Hay que tener en cuenta que si está activado el recorte de caras traseras, al dibujar el cubo del fondo no se verá nada en la pantalla. El algoritmo decide qué caras eliminar según el sentido de giro (*winding*) de los vértices del triángulo proyectado en la pantalla, no según las normales: las caras de la primitiva del cubo están armadas para verse de frente desde afuera y, como la escena queda contenida *dentro* del volumen del cubo, desde el interior todas se ven con el giro invertido y el algoritmo las elimina. Para evitar esto, en este objeto particular, hay tres soluciones: desactivar el recorte de caras traseras para dibujar el cubo y luego volver a activarlo (es lo que hace el fragmento de código del método (a)), recortar las caras delanteras en lugar de las traseras durante ese dibujo (`glCullFace(GL_FRONT)`), o invertir el orden de los índices de cada triángulo del cubo.

Parte 5 - El problema de la altitud

Mientras el observador (cámara) se mantenga a la misma altura que el terreno y este último sea suave, el horizonte del cubemap siempre se mantendrá muy cercano al límite del terreno y no se notarán defectos visuales (se puede mejorar la sensación de profundidad agregando un efecto niebla suave). Pero en el caso de una escena para un simulador de vuelo, cuando el observador (aeronave) comienza a ascender y alejarse del terreno, se presenta un defecto visual en el horizonte ya que se empieza a ver la mitad inferior del cubemap y se pierde la coherencia de la escena.

Esto se debe a que el fondo sigue al observador y el terreno, en cambio, se mantiene en una posición fija y es además una malla de tamaño finito (ventana de terreno). Como el horizonte pintado en el cubemap queda siempre a la altura del ojo, el borde del terreno más alejado del observador comienza a "descender" respecto al horizonte a medida que aumenta la altura y entre los dos aparece una banda que corresponde a la textura del cubemap por debajo del horizonte (y eventualmente parte de la cara inferior también).

Para analizar cómo se puede resolver este problema, se plantea qué relación existe entre el tamaño de la ventana del terreno, la altitud del observador y el tamaño de la banda que se observa entre el horizonte y el borde del terreno. Con el ojo sobre el centro de una ventana de terreno de lado L y a una altitud h, el borde más alejado del terreno produce una banda por debajo del horizonte medida como desplazamiento angular por $\arctan\left(h/(L/2)\right)$:

altitud	terreno de 2 km	terreno de 20 km	horizonte real
100 m	5,71°	0,57°	0,32°
500 m	26,57°	2,86°	0,72°
1000 m	45,00°	5,71°	1,02°
10000 m	84,29°	45,00°	3,21°

La última columna de la tabla representa cuánto baja el horizonte real debido a la curvatura de la Tierra. Con un terreno de 2 km baja entre 18 y 44 veces menos que la banda y con uno de 20 km entre 1,8 y 14 veces menos, por lo que la curvatura no es la causa del problema. La causa es que el terreno es finito y no llega al horizonte. El problema es que no se resuelve agrandando la malla uniforme del Práctico 07: para que el borde llegue al horizonte real visto desde una altitud de 1000 m, la ventana de terreno tendría que medir más de 100 km de lado (con celdas de 10 m es más de 3000 veces la cantidad de celdas de la ventana de 2 km).

Una solución simple: continuar el suelo en el fondo

Los simuladores de vuelo resuelven este problema con técnicas más sofisticadas: el terreno se extiende realmente hasta el horizonte y se dibuja con menos detalle a medida que se aleja del observador (niveles de detalle, LOD por *level of detail*). Para este práctico se propone una solución más simple que aprovecha el dibujo del fondo: los píxeles del fondo que quedan por debajo del horizonte se pintan como si fueran parte de un terreno plano e infinito que continúa al terreno de la malla usando la misma textura y la misma escala. Como el terreno se dibuja después que el fondo, ese terreno ficticio sólo queda a la vista donde la malla no llega, es decir, más allá del borde de la ventana de terreno.

Cómo es el cálculo

Cada píxel del fondo corresponde a una dirección $\vec{d}$ (la misma que se usa para muestrear el cubemap). Si desde la posición del observador $\vec{O}$ se traza un rayo en esa dirección, sus puntos son $\vec{P}(t) = \vec{O} + t\,\vec{d}$, donde t es la distancia recorrida desde el observador (con $\vec{d}$ normalizada). El terreno es un plano horizontal $y = y_s$ (por ejemplo, a la altura media del terreno), por lo que el rayo lo corta cuando $O_y + t\,d_y = y_s$, es decir, para:

$$t = \frac{O_y - y_s}{-d_y}$$

Este valor es positivo sólo si $d_y < 0$, que es justamente la condición para que la dirección esté por debajo del horizonte; por encima del horizonte el rayo nunca corta el suelo y el píxel se pinta con el cubemap como en la Parte 4. Conocido t se obtiene el punto del terreno que se ve en ese píxel como $(x, z) = (O_x + t\,d_x,\ O_z + t\,d_z)$ y al dividirlo por los metros que cubre una tesela en el terreno se obtienen las coordenadas (u, v) para leer la textura del suelo con la misma escala que la malla.

Hacia el horizonte d_y tiende a cero y t crece sin límite: los puntos del suelo están cada vez más lejos y las coordenadas (u, v) cambian muy rápido de un píxel al siguiente. Esto no es un problema porque el filtrado con mipmaps detecta ese cambio y usa niveles cada vez más chicos de la textura, hasta llegar a su color promedio, que es lo que se espera ver en la lejanía.

En el siguiente fragmento de código se muestra una propuesta del *fragment* shader para el fondo con el suelo continuado. El *vertex* shader es el mismo de la Parte 4.

```
// background-suelo.fs --- mismo background.vs de la Parte 4
#version 460 core
in vec3 vDireccion;

layout (binding = 0) uniform samplerCube uCielo;
layout (binding = 1) uniform sampler2D   uTexturaSuelo;   // la textura del terreno

uniform vec3  uOjo;            // posicion del observador en el mundo
uniform float uAlturaSuelo;    // altura del plano que continua el terreno
uniform float uMetrosTesela;   // metros que cubre una tesela en el terreno

out vec4 FragColor;

void main()
{
    vec3 d = normalize(vDireccion);

    // Rayo P(t) = uOjo + t d contra el plano y = uAlturaSuelo:
    //   uOjo.y + t d.y = uAlturaSuelo   ->   t = (uOjo.y - uAlturaSuelo) / (-d.y)
    float t     = (uOjo.y - uAlturaSuelo) / max(-d.y, 1.0e-4);
    vec2  xz    = uOjo.xz + t * d.xz;                          // punto del terreno visto
    vec4  suelo = texture(uTexturaSuelo, xz / uMetrosTesela); // misma escala que la malla
    vec4  cielo = texture(uCielo, vDireccion);

    // Por debajo del horizonte el color es el terreno continuado; por encima, el cielo.
    FragColor = (d.y < 0.0) ? suelo : cielo;
}
```

Tener en cuenta

- Las dos texturas se leen siempre, también la del terreno para los píxeles que están por encima del horizonte, y recién al final se elige una. El filtrado con mipmaps calcula cuánto cambian las coordenadas entre píxeles vecinos y para eso todos tienen que hacer la misma lectura; dentro de un `if` ese cálculo no está definido. El `max(-d.y, 1.0e-4)` evita la división por cero en el horizonte.
- El shader necesita la posición del observador (cámara) en el mundo (`uOjo`), que la matriz de vista sin traslación ya no tiene, por lo que hay que pasarla aparte.
- Para que la textura del suelo continúe la de la malla sin un salto en el borde, las dos tienen que usar la misma escala y el mismo origen. Por ejemplo, con la malla ubicada en el origen del mundo y 40 teselas sobre 2 km, la relación `uMetrosTesela` vale 50 [m/tesela].

Limitaciones

La solución funciona bien mientras el terreno sea suave cerca del borde de la ventana de terreno. El plano continúa la textura pero no la forma, por lo que si hay relieve pronunciado en el borde (montañas, por ejemplo), éste aparece cortado con una llanura detrás. En ese caso hace falta que el borde quede lo suficientemente lejos como para que no se distinga, lo que vuelve a requerir una ventana de terreno más grande o, directamente, las técnicas de niveles de detalle mencionadas antes. Por otra parte, la escena mantiene un horizonte con un salto de color entre el terreno y el cielo. Lo que termina de cerrar el aspecto de la escena es la niebla en la lejanía (el terreno y el suelo se funden con el color del horizonte del cielo), que es tema de la Unidad IX y no hace falta implementarla en este práctico.

Implementación y verificación

- Subir con la cámara hasta que aparezca la banda y anotar a qué altitud empieza a notarse con la ventana de terreno propia. Comparar con el valor que da la fórmula.
- Implementar el fondo con el suelo continuado y verificar que la banda desaparece a cualquier altitud. Mirar hacia el borde de la ventana de terreno y verificar que no se distingue dónde termina la malla.

Opcional

El cielo pintado por elevación.
En lugar de usar una textura tipo *cubemap* (que tiene seis imágenes) para pintar el fondo se puede usar una textura **1D** con un degradé de colores del horizonte al cenit, indexada por la elevación de la dirección. Con esto se puede lograr un cielo con un degrade continuo en todo el ancho de la pantalla. Como geometría se puede usar el mismo cubo y el siguiente *fragment shader* en lugar de `background.fs`.

```
// background-degrade.fs --- mismo background.vs de la Parte 4
#version 460 core
in vec3 vDireccion;

layout (binding = 0) uniform sampler1D uDegrade;   // s = 0 horizonte, 1 cenit

out vec4 FragColor;

void main()
{
    float elevacion = asin(normalize(vDireccion).y);        // [-pi/2, pi/2]
    float s = clamp(elevacion / radians(90.0), 0.0, 1.0);   // abajo: s = 0
    FragColor = texture(uDegrade, s);
}
```

Mientras que la textura 1D se puede crear con el siguiente conjunto de llamadas de OpenGL.

```
// La textura 1D: 64 texels RGBA en el arreglo 'colores'
GLuint degrade;
glCreateTextures(GL_TEXTURE_1D, 1, &degrade);
glTextureStorage1D(degrade, 1, GL_RGBA8, 64);                 // 64 texels
glTextureSubImage1D(degrade, 0, 0, 64, GL_RGBA, GL_UNSIGNED_BYTE, colores);
glTextureParameteri(degrade, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTextureParameteri(degrade, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTextureParameteri(degrade, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
```

Un cielo despejado ocupa 64 texels contra los 18 MiB de un cubemap de 1024×1024, y además permite cambiar la hora del día con sólo cambiar los colores de la textura. Por debajo del horizonte, el `clamp` hace que todo salga del color del texel 0, por lo que sin el suelo continuado de la Parte 5 la banda queda pintada de un color liso. Para combinar las dos cosas, el shader del fondo de la Parte 5 puede usar el degradé en lugar del cubemap para los píxeles que están por encima del horizonte.

La desventaja de la textura 1D es que genera un fondo relativamente simple que va a requerir agregar otras capas de objetos para mejorar la apariencia de la escena.

Algunas cuestiones a pensar

- La función `stbi_set_flip_vertically_on_load` modifica un estado global de la biblioteca y no es un parámetro de cada carga. Si queda en 1 después de leer la textura del terreno, ¿qué le pasa al cielo que se lee después? ¿Qué pasa si el `ResourceManager` guarda las imágenes en caché por clave y la misma imagen se pide una vez invertida y otra no?
- Con `GL_REPEAT` y `GL_MIRRORED_REPEAT` un damero se ve idéntico. ¿Por qué? ¿Qué tiene que tener una imagen de prueba para poder distinguirlos?
- Un cubemap de 1024×1024 ocupa unas diez veces más memoria que el terreno entero (malla y textura incluidas). ¿Vale la pena generarle mipmaps? ¿En qué caso un texel del cielo se ve reducido en la pantalla?
- ¿Qué pasa si se dibuja el fondo con la matriz de vista completa, sin quitarle la traslación? Conviene predecir el resultado antes de probarlo.
- Cuando se incorpore la iluminación difusa, el brillo del terreno va a depender de la orientación de cada triángulo. ¿Qué hay que agregar en el shader del fondo para que el suelo continuado no se distinga de la malla? ¿Qué normal tiene ese plano?
- El heightmap del Práctico 07 es una grilla de alturas indexada por (i, j). ¿Qué haría falta para guardarlo como una textura y qué ganaría el programa con eso?

Cómo se sabe que está bien

- La imagen de prueba asimétrica se ve derecha en el terreno y en las caras del cubo, sin estar invertida ni espejada.
- Hacia el horizonte el terreno no muestra franjas ni parpadea al mover la cámara (si parpadea, faltan los mipmaps o el filtro de reducción no los usa).
- Al trasladar la cámara el fondo no se mueve y al rotarla sí. No se distinguen las aristas del cubo.
- El fondo nunca tapa al terreno ni a la aeronave, desde ninguna de las vistas.
- A ninguna altitud aparece la banda entre el borde del terreno y el horizonte, y mirando hacia el borde de la ventana de terreno no se distingue dónde termina la malla.
- Ningún objeto se ve negro ni de un color que no está en la imagen.
- Están escritas la cantidad de teselas y la altitud a la que empieza a notarse la banda sin el suelo continuado.

Como en los prácticos anteriores, las interfaces mostradas son un **punto de partida y no el molde**. Las decisiones de adentro son de cada uno y hay que poder explicarlas.

El **checkpoint del proyecto** está planificado para el **viernes 27 de noviembre**, y el cielo es parte del requisito 1b.